

# TP20 — Script Python pour déployer une policy SD-WAN via API

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

## Description

Ce TP conclut le projet fil rouge : après avoir formé la topologie SD-WAN (TP19), on déploie une policy centralisée — une règle de priorisation de trafic (le trafic VN\_CORP passe en priorité haute, VN\_GUEST en priorité basse) — sur l'ensemble des sites, avec le même cycle asynchrone que DNA Center (TP15) : création, tâche de suivi, activation confirmée. Une policy ciblant un site absent de l'overlay est rejetée avant même toute tentative d'activation.

## Prérequis

- TP1 à TP19 validés

## Étapes

### Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour5-tp20
```

### Étape 2 — Étendre le mock vManage avec les politiques centralisées

```
cat > scripts/mock_vmanage_api.py << 'EOF'
import secrets
import uuid

from flask import Flask, request, jsonify, session

app = Flask(__name__)
app.secret_key = "dev-secret-key-not-for-production"

VALID_USER = "admin"
VALID_PASS = "vmanagepass123"
```

```

DEVICES = [
    {"deviceId": "vedge-siege", "host-name": "siege-router", "site-id": None, "rea
    {"deviceId": "vedge-agence1", "host-name": "agence1-router", "site-id": None, "
    {"deviceId": "vedge-agence2", "host-name": "agence2-router", "site-id": None, "
]

XSRF_TOKENS = {}
POLICIES = {}
TASKS = {}

def find_device(device_id):
    return next((d for d in DEVICES if d["deviceId"] == device_id), None)

def configured_site_ids():
    return {d["site-id"] for d in DEVICES if d["site-id"]}

def check_session():
    return session.get("authenticated") is True

def check_xsrf():
    token = request.headers.get("X-XSRF-TOKEN")
    return token and XSRF_TOKENS.get(session.get("sid")) == token

def check_auth():
    return check_session() and check_xsrf()

@app.route("/j_security_check", methods=["POST"])
def j_security_check():
    username = request.form.get("j_username")
    password = request.form.get("j_password")
    if username != VALID_USER or password != VALID_PASS:
        return "Login failed", 401
    session["authenticated"] = True
    session["sid"] = secrets.token_hex(8)
    return "", 200

@app.route("/dataservice/client/token", methods=["GET"])
def client_token():
    if not check_session():
        return jsonify({"error": "Not authenticated"}), 401

```

```

    token = secrets.token_hex(16)
    XSRF_TOKENS[session["sid"]] = token
    return token, 200

@app.route("/dataservice/system/device/vedges", methods=["GET"])
def list_vedges():
    if not check_auth():
        return jsonify({"error": "Missing or invalid session/XSRF token"}), 401
    return jsonify({"data": DEVICES})

@app.route("/dataservice/system/device/site", methods=["POST"])
def configure_site():
    if not check_auth():
        return jsonify({"error": "Missing or invalid session/XSRF token"}), 401
    body = request.get_json(force=True)
    device = find_device(body.get("deviceId"))
    if not device:
        return jsonify({"error": f"Device inconnu : {body.get('deviceId')}"}), 404
    device["site-id"] = body.get("siteId")
    device["tloc-color"] = body.get("tlocColor")
    return jsonify({"status": "success", "device": device})

@app.route("/dataservice/device/omp/summary", methods=["GET"])
def omp_summary():
    if not check_auth():
        return jsonify({"error": "Missing or invalid session/XSRF token"}), 401
    device_id = request.args.get("deviceId")
    device = find_device(device_id)
    if not device:
        return jsonify({"error": f"Device inconnu : {device_id}"}), 404
    if not device["site-id"]:
        return jsonify({"data": {"omp-peers-up": 0, "status": "site non configure"}})
    configured_others = [d for d in DEVICES if d["site-id"] and d["deviceId"] != device_id]
    return jsonify({"data": {"omp-peers-up": len(configured_others), "status": "up"}})

@app.route("/dataservice/template/policy/vsmart", methods=["POST"])
def create_policy():
    if not check_auth():
        return jsonify({"error": "Missing or invalid session/XSRF token"}), 401
    body = request.get_json(force=True)
    policy_name = body.get("policyName")
    target_sites = set(body.get("targetSites", []))
    rules = body.get("rules", [])

```

```

    if not policy_name or not target_sites or not rules:
        return jsonify({"error": "policyName, targetSites et rules sont requis"}),

    missing = target_sites - configured_site_ids()
    if missing:
        return jsonify({"error": f"Site(s) absent(s) de l'overlay SD-WAN, impossible"}),

    policy_id = f"policy-{uuid.uuid4().hex[:8]}"
    process_id = f"proc-{uuid.uuid4().hex[:8]}"
    POLICIES[policy_id] = {
        "policyName": policy_name,
        "targetSites": sorted(target_sites),
        "rules": rules,
        "active": False,
    }
    TASKS[process_id] = {"policyId": policy_id, "polls": 0}
    return jsonify({"policyId": policy_id, "processId": process_id})

@app.route("/dataservice/device/action/status/<process_id>", methods=["GET"])
def action_status(process_id):
    if not check_auth():
        return jsonify({"error": "Missing or invalid session/XSRF token"}), 401
    if process_id not in TASKS:
        return jsonify({"data": {"status": "unknown"}}), 404

    task = TASKS[process_id]
    task["polls"] += 1
    if task["polls"] < 3:
        return jsonify({"data": {"status": "IN_PROGRESS"}})

    POLICIES[task["policyId"]]["active"] = True
    return jsonify({"data": {"status": "Success"}})

@app.route("/dataservice/template/policy/vsmart", methods=["GET"])
def list_policies():
    if not check_auth():
        return jsonify({"error": "Missing or invalid session/XSRF token"}), 401
    return jsonify({"data": [{"policyId": pid, **p} for pid, p in POLICIES.items()]})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000, debug=False)
EOF

```

Point métier clé : create\_policy vérifie que tous les targetSites demandés font bien partie de l'overlay déjà formé (configured\_site\_ids())

— impossible de déployer une policy sur un site qui n'existe pas encore dans la topologie SD-WAN.

### Étape 3 — Ajouter les méthodes de policy au client

```
cat >> scripts/netops_client/vmanage_client.py << 'EOF'

def create_policy(self, policy_name, target_sites, rules):
    url = f"{self.base_url}/dataservice/template/policy/vsmart"
    body = {"policyName": policy_name, "targetSites": target_sites, "rules": rules}
    response = self.session.post(url, headers=self._headers(), json=body, timeout=self.timeout)
    if response.status_code != 200:
        raise VManageError(f"Echec de la creation de la policy : {response.text}")
    return response.json()

def wait_for_action(self, process_id, max_attempts=6, delay=1):
    import time
    url = f"{self.base_url}/dataservice/device/action/status/{process_id}"
    for _ in range(max_attempts):
        response = self.session.get(url, headers=self._headers(), timeout=self.timeout)
        if response.status_code != 200:
            raise VManageError(f"Erreur de suivi d'activation : {response.text}")
        status = response.json()["data"]["status"]
        if status != "IN_PROGRESS":
            return status
        time.sleep(delay)
    raise VManageError(f"Activation {process_id} toujours en cours apres {max_attempts} tentatives")

def list_policies(self):
    url = f"{self.base_url}/dataservice/template/policy/vsmart"
    response = self.session.get(url, headers=self._headers(), timeout=self.timeout)
    if response.status_code != 200:
        raise VManageError(f"Erreur de listage des policies : {response.text}")
    return response.json()["data"]

EOF
```

### Étape 4 — Créer le script de déploiement de la policy

```
cat > scripts/deploy_sdwan_policy.py << 'EOF'
import os
import sys

from netops_client import VManageClient, VManageError

VMANAGE_URL = os.environ.get("VMANAGE_URL", "http://localhost:8000")
VMANAGE_USER = os.environ.get("VMANAGE_USER", "admin")
VMANAGE_PASS = os.environ.get("VMANAGE_PASS")
```

```

if not VMANAGE_PASS:
    print("ERREUR : variable d'environnement VMANAGE_PASS non definie.")
    sys.exit(1)

SITE_TOPOLOGY = [
    ("vedge-siege", 100, "mpls"),
    ("vedge-agence1", 101, "biz-internet"),
    ("vedge-agence2", 102, "biz-internet"),
]

POLICY_NAME = "Policy_QoS_Corp"
POLICY_RULES = [
    {"action": "set-priority-high", "sourceVN": "VN_CORP", "destVN": "VN_CORP"},
    {"action": "set-priority-low", "sourceVN": "VN_GUEST", "destVN": "VN_GUEST"},
]
TARGET_SITES = [100, 101, 102]

def main():
    client = VManageClient(base_url=VMANAGE_URL, username=VMANAGE_USER, password=VMANAGE_PASS)
    client.login()
    print("Authentifie sur vManage.\n")

    print("=== Verification/formation de la topologie SD-WAN ===")
    for device_id, site_id, tloc_color in SITE_TOPOLOGY:
        client.configure_site(device_id, site_id, tloc_color)
        print(f"  {device_id} -> site-id {site_id}, TLOC {tloc_color}")
    print()

    print(f"=== Deploiement de la policy centralisee '{POLICY_NAME}' ===")
    print(f"  Sites cibles : {TARGET_SITES}")
    for rule in POLICY_RULES:
        print(f"  Regle : {rule['sourceVN']} -> {rule['destVN']} : {rule['action']}")

    try:
        result = client.create_policy(POLICY_NAME, TARGET_SITES, POLICY_RULES)
    except VManageError as e:
        print(f"\n ECHEC : {e}")
        sys.exit(1)

    policy_id, process_id = result["policyId"], result["processId"]
    print(f"\n  Policy creee : {policy_id}, activation en cours (tache {process_id})")

    status = client.wait_for_action(process_id)
    print(f"  Statut final de l'activation : {status}\n")

    if status != "Success":
        print("ECHEC : la policy n'a pas ete activee correctement.")

```

```

        sys.exit(1)

    print("=== Verification finale ===")
    policies = client.list_policies()
    for p in policies:
        if p["policyId"] == policy_id:
            print(f"  {p['policyName']} : active = {p['active']}, sites = {p['targetSites']}")

    print("\nPolicy SD-WAN deployee et active avec succes sur tous les sites.")

if __name__ == "__main__":
    try:
        main()
    except VManageError as e:
        print(f"Erreur vManage : {e}")
        sys.exit(1)
EOF

```

## Étape 5 — Tester le déploiement complet

```

nohup python scripts/mock_vmanage_api.py > /tmp/vmanage.log 2>&1 &
sleep 2
export VMANAGE_PASS=vmanagepass123
python scripts/deploy_sdwan_policy.py

```

### Résultat attendu :

Authentifie sur vManage.

```

=== Verification/formation de la topologie SD-WAN ===
vedge-siege -> site-id 100, TL0C mpls
vedge-agence1 -> site-id 101, TL0C biz-internet
vedge-agence2 -> site-id 102, TL0C biz-internet

```

```

=== Deploiement de la policy centralisee 'Policy_QoS_Corp' ===
Sites cibles : [100, 101, 102]
Regle : VN_CORP -> VN_CORP : set-priority-high
Regle : VN_GUEST -> VN_GUEST : set-priority-low

```

```

Policy creee : policy-1f29659f, activation en cours (tache proc-
d6d3c076)...
Statut final de l'activation : Success

```

```

=== Verification finale ===
Policy_QoS_Corp : active = True, sites = [100, 101, 102]

```

Policy SD-WAN deployee et active avec succes sur tous les sites.

**Si vous obtenez une erreur 404 Not Found (page HTML Flask générique) :** un ancien mock vManage (du TP19, sans les routes de policy) tourne probablement encore sur le port 8000. Libérez-le et relancez :

```
netstat -ano | grep ":8000" | grep LISTENING | awk '{print $5}' | sort -u | wh
nohup python scripts/mock_vmanage_api.py > /tmp/vmanage.log 2>&1 &
sleep 2
```

## Étape 6 — Tester le rejet d’une policy ciblant un site inexistant

Modifiez temporairement `TARGET_SITES = [100, 101, 102]` en `TARGET_SITES = [100, 101, 102, 999]`, redémarrez le mock, puis relancez :

```
python scripts/deploy_sdwan_policy.py
```

### Résultat attendu :

ECHEC : Echec de la creation de la policy : {"error":"Site(s) absent(s) de l'overlay S  
WAN, impossible de deployer la policy : [999]"}  
Le rejet se fait avant même la création d’une tâche d’activation.

Remettez `TARGET_SITES = [100, 101, 102]` ensuite.

## Étape 7 — Committer et pousser

```
git add scripts/mock_vmanage_api.py scripts/netops_client/vmanage_client.py scripts
git commit -m "TP20 : deploiement d'une policy SD-WAN centralisee via API"
git push -u origin <prenom>-<nom>-jour5-tp20
```

Ouvrir une pull request `<prenom>-<nom>-jour5-tp20` → dev.

## Comprendre le TP

Le cycle création → tâche → polling → vérification revient une dernière fois, cette fois pour une policy plutôt qu’un template (TP15) — ce n’est pas un détail propre à DNA Center, mais un vrai standard des API réseau modernes : toute action qui prend du temps à se propager sur de vrais équipements est traitée de façon asynchrone.

La validation métier avant l’action (vérifier que les sites cibles sont bien dans l’overlay avant de créer la policy) est délibérément placée avant la création de la tâche asynchrone, pas après son activation — inutile de lancer un déploiement voué à l’échec si on peut détecter le problème immédiatement. C’est le même réflexe que les pre-checks Ansible du TP11, appliqué ici à une API cloud.

## Validation

- ☐ `python scripts/deploy_sdwan_policy.py` forme la topologie, déploie la policy, et confirme son activation (`active = True`)
- ☐ Une policy ciblant un site inexistant est rejetée immédiatement, avec un message clair identifiant le site en cause



☐ Une pull request <prenom>-<nom>- jour5-tp20 → dev est ouverte